

Capstone Design 2026 PROJECT
7팀

Q&A 마스터 레퍼런스

LLM 추천 패키지 공급망 공격 탐지 도구

발표 후 질의응답 대비 — 전 기능/파일/데이터 정리본

작성일: 2026-05-19

7팀 캡스톤 Q&A 마스터 레퍼런스

발표 후 질의응답 대비. 모든 기능-파일-데이터를 코드 기준으로 누락 없이 정리. 전문 용어는 첫 등장 시 괄호로 풀어쓴다.

0. 한 줄 정의

pkgsentinel은 **PyPI(파이썬 패키지 저장소) / npm(자바스크립트 패키지 저장소)의 패키지를 설치하지 않고 정적 분석으로 검사하는 도구다.** LLM(거대언어모델) 환각으로 만들어진 허위 패키지, 타이포스쿼팅(오타 변종) 패키지, 정상 패키지의 변조본을 탐지 대상으로 한다.

1. 위협 모델

1.1 슬롭스쿼팅(Slopsquatting)

LLM 환각으로 존재하지 않는 패키지명이 반복 추천되면, 공격자가 그 이름으로 악성 패키지를 사전 등록 -> 개발자가 AI 추천을 믿고 설치 -> 악성 코드가 코드베이스에 유입.

1.2 인접 위협 - 같은 도구로 함께 다름

- 타이포스쿼팅(Typosquatting): `requests` -> `requets`, `numpy` -> `nunpy` 같은 오타 변종
- 콤보스쿼팅(Combosquatting): `requests-utils` 같은 단어 결합형
- 호모글리프(Homoglyph): `0` ↔ `o`, `1` ↔ `l` 같이 모양만 비슷한 글자 사용
- 사후 변조: 정상 패키지의 메인테이너 계정이 탈취되어 어느 시점부터 악성으로 바뀌는 케이스 (event-stream, ua-parser-js, colors.js 사건)
- 의존성 혼동(Dependency Confusion): 사내 패키지명과 같은 이름을 퍼블릭 레지스트리에 더 높은 버전으로 올리는 공격

1.3 비대상

- 바이너리 단계 안티바이러스 (.exe/.dll 패턴 매칭) - 우리는 소스 정적분석
- 일반 웹 취약점 (SQLi, XSS 등) - 우리는 공급망 공격 한정
- 인기도/다운로드 수에 따른 신뢰 할인 - 인기 패키지도 풀 파이프라인

2. 5단계 판정 체계 (Verdict)

`src/pkgsentinel/schema.py` 의 `Verdict` Enum.

Verdict	의미	결정 조건 (verdict_rules.py 기준)
MALICIOUS	악성 확정	고심각 TTP(공격 기법) 매칭 $\geq 1 + \text{LLM "malicious" + 악성 evidence 중 최대 신뢰도} \geq 0.85$
HIGH_RISK	높은 위험	강한 TTP 매칭 또는 버전 차이 위험 + LLM in {suspicious, malicious}
SUSPICIOUS	의심	약한 TTP 매칭(LLM!=BENIGN) 또는 버전 차이 있음 또는 LLM 의심 ≥ 2 건
AGENTIC	에이전트 패키지	AISLOPSQ Manifest(LLM 에이전트 메타데이터 표준) 매칭, 악성 아님
CLEAN	깨끗	모든 스테이지 통과 + 의심 근거 없음 (또는 약한 근거 + LLM=BENIGN)
ERROR	분석 실패	Stage 2, 4, 5(필수 스테이지) 중 하나 이상 실패
CANNOT_ANALYZE	분석 불가	레지스트리에 등록되지 않은 이름 (소스 부재)

2.1 핵심 임계값 (verdict_rules.py)

상수	값	의미
MALICIOUS_CONFIDENCE_THRESHOLD	0.85	MALICIOUS 승격 임계값. 악성 evidence 의 max 신뢰도 기준(구버전은 전체 평균이라 저신뢰 잡음 1건에 강등됐음 - H-6에서 max로 교정)
STRONG_MATCH_SIMILARITY	0.85	강한 TTP 매칭 코사인 유사도

상수	값	의미
WEAK_MATCH_SIMILARITY	0.70	약한 TTP 매칭 코사인 유사도
LLM_SUSPICIOUS_QUORUM	2	LLM-only SUSPICIOUS 승격에 필요한 의심 evidence 개수 (단일 알람 폭주 억제)

2.2 판정 정책 (LLM BENIGN 우선)

LLM이 BENIGN(정상)으로 본 약한 매칭은 SUSPICIOUS로 올리지 않는다. 강한 매칭(유사도 ≥ 0.85 , severity != LOW)은 LLM 판정과 무관하게 HIGH_RISK/MALICIOUS 분기로 진입.

3. Evidence(판정 근거) 데이터 모델

모든 verdict는 점수가 아닌 **Evidence** 리스트로만 설명된다.

```
Evidence 한 건:
└ 어디서 발견했는가
  | file_path / line_start / line_end / code_snippet
└ 무엇이 의심스러운가 (행위)
  | behavior_sequence (예: ["os.environ.get", "base64.b64encode", "requests.post"])
  | attack_dimensions (4 차원 중 어디에 속하는가)
└ 공신력 있는 근거 매핑
  | ttp_id (MITRE 공격 기법 코드, 예 "T1048.003")
  | ttp_name, ttp_source (MITRE ATT&CK/ATLAS/OWASP LLM 중)
  | vector_similarity (지식DB와의 코사인 유사도, 0~1)
  | ttp_severity (HIGH/MEDIUM/LOW)
└ LLM 재검토
  | llm_verdict (malicious/suspicious/benign)
  | llm_reasoning, llm_model
└ 버전 변화 (해당 시)
  | version_diff: VersionDiffInfo
└ 종합 신뢰도 confidence (0~1)
```

4. 4 공격 차원 (Attack Dimension)

Cerebro 논문(공급망 패키지 공격 분석 학술 논문, 2024) 기준 네 종류로 행위를 분류.

차원	의미	예
INFORMATION_READING	정보 읽기	os.environ.get, fs.readFile, Path.home(), process.env
ENCODING	인코딩/난독화	base64.b64decode, bytes.fromhex, compile(), Buffer.from
PAYLOAD_EXECUTION	페이로드 실행	exec, eval, subprocess.run, child_process.exec, Function()
DATA_TRANSMISSION	데이터 송신	requests.post, socket.connect, fetch, http.request

이 네 차원의 **호출 순서**가 6 시퀀스 패턴(섹션 6)의 기본 단위.

5. 49개 악성 지표 카탈로그

[src/pkgsentinel/knowledge/malicious_indicators.py](#).

근거 논문: *Unveiling Malicious Logic: Towards a Statement-Level Taxonomy and Dataset for Securing Python Packages* (arXiv 2025).

7 카테고리 × 47 베이스 + 2 다운로드 패턴 = **49 지표 분류 체계**. 각 지표는 정규식 또는 AST(추상구문트리, 코드를 트리 구조로 표현한 것) 기반 매칭으로 탐지.

****정확한 수치 (발표 시 주의)**:** 카탈로그는 ****49종****, 그중 `indicator\_matcher.py`에 실제 매치가 구현된 것은 ****36종****. 나머지 13종(EXF-002, SYS-006/008, NET-003~006, DEF-001/002/004, MET-002/005/006)은 분류 체계에만 존재하고 매치 미구현. "49개를 전부 탐지"가 아니라 "49종 분류 체계 + 36종 활성 매치"로 답할 것.

5.1 EXS - Execution Stage (실행 단계, 3개)

코드	이름	심각도	설명	MITRE TTP
EXS-001	Import-Time Execution	HIGH	모듈 import 즉시 자동 실행	T1059
EXS-002	Install-Time Execution	HIGH	setup.py 최상위 코드, 설치 시 자동 실행	T1195.002
EXS-003	Lifecycle Hook Hijack	HIGH	setuptools install/develop 혹은 오버라이드	T1546

5.2 EXM - Execution Mechanism (실행 방식, 8개)

코드	이름	심각도	설명	MITRE TTP
EXM-001	Dynamic Evaluation	HIGH	eval/exec 로 동적 코드 실행	T1059.006
EXM-002	Conditional Payload Trigger	MEDIUM	OS/시간 조건부 발동 (분석 회피)	T1480
EXM-003	Binary Execution	HIGH	번들 컴파일 바이너리 실행	T1027.002
EXM-004	Hidden Code Execution	HIGH	백그라운드/숨김 서브프로세스	T1564
EXM-005	Dynamic Module Import	MEDIUM	변수로 모듈명 조립 후 import	T1027
EXM-006	Dynamic Package Install	HIGH	런타임 pip install	T1105
EXM-007	Script File Execution	HIGH	번들 .sh/.ps1/.py 실행	T1059
EXM-008	Shell Command Execution	HIGH	os.system, shell=True	T1059.004

5.3 EXF - Exfiltration (탈취 송신, 5개)

코드	이름	심각도	설명	MITRE TTP
EXF-001	Data Exfiltration	HIGH	자격증명/API 키 외부 송신	T1552, T1048
EXF-002	File Exfiltration	HIGH	파일 전체 외부 송신	T1041
EXF-003	DNS Tunneling	HIGH	DNS 쿼리로 데이터 송신	T1071.004
EXF-004	Webhook Exfiltration	HIGH	Slack/Discord/Telegram 채팅 API 악용	T1567.002
EXF-005	Suspicious Domain Exfiltration	HIGH	pastebin/transfer.sh/.onion	T1041

5.4 SYS - System Impact (시스템 영향, 9개)

코드	이름	심각도	설명	MITRE TTP
SYS-001	Environment Modification	HIGH	PATH/LD_PRELOAD 변경	T1574
SYS-002	Startup File Persistence	HIGH	.bashrc/레지스트리/crontab	T1547
SYS-003	Crypto Wallet Harvesting	HIGH	지갑 파일 스캔 (wallet.dat, MetaMask)	T1005
SYS-004	Directory Enumeration	MEDIUM	os.walk/glob 으로 파일 열거	T1083
SYS-005	System Info Reconnaissance	MEDIUM	uname/getuser/hostname	T1082
SYS-006	File Relocation	MEDIUM	영구화 위치로 파일 이동	T1564
SYS-007	File Deletion	MEDIUM	흔적 제거용 삭제	T1070.004
SYS-008	Arbitrary File Write	MEDIUM	임의 파일 작성	T1105
SYS-009	Sensitive Path Write	HIGH	/etc, System32 등에 쓰기	T1105

5.5 NET - Network Operations (네트워크, 10개)

코드	이름	심각도	설명	MITRE TTP
NET-001	Geolocation Lookup	MEDIUM	ipinfo.io 등 위치 조회	T1614
NET-002	Mining Pool Connection	HIGH	stratum+tcp 마이닝 풀	T1496
NET-003	Suspicious Connection	HIGH	공격자 통제 또는 신규 도메인	T1071
NET-004	Archive Dropper	HIGH	압축 아카이브 다운+해제	T1105
NET-005	Binary Dropper	HIGH	.exe/.dll/.so 다운	T1105
NET-006	Payload Dropper	HIGH	urllopen + exec 조합	T1105
NET-007	Script Dropper	HIGH	curl W	bash 패턴
NET-008	Reverse Shell	HIGH	역방향 셸 연결	T1059

코드	이름	심각도	설명	MITRE TTP
NET-009	SSL Validation Bypass	MEDIUM	verify=False, rejectUnauthorized:false	T1573
NET-010	Unencrypted Communication	MEDIUM	http:// 평문 통신	T1071.001

5.6 DEF - Defense Evasion (방어 회피, 6개)

코드	이름	심각도	설명	MITRE TTP
DEF-001	ASCII Art Deception	MEDIUM	큰 ASCII 아트로 악성 코드 가림	T1027
DEF-002	Computational Obfuscation	MEDIUM	비트연산/문자열 연산 난독화	T1027
DEF-003	Encoding-Based Obfuscation	HIGH	base64/hex 인코딩	T1027.005, T1140
DEF-004	Encryption-Based Obfuscation	HIGH	AES/Fernet/XOR 암호화	T1027
DEF-005	Embedded String Payload	HIGH	문자열에 코드 임베드 후 실행	T1027
DEF-006	Error Suppression	LOW	try/except: pass, 2>/dev/null	T1564

5.7 MET - Metadata Manipulation (메타데이터 조작, 6개)

코드	이름	심각도	설명	MITRE TTP
MET-001	Suspicious Author Identity	MEDIUM	일회용 이메일/placeholder 저자	T1195.001
MET-002	Combosquatting	MEDIUM	유명 패키지명 + 접미사	T1195.001
MET-003	Suspicious Dependency	MEDIUM	목적 불일치 의존성 선언	T1195.001
MET-004	Description Anomaly	LOW	키워드 스테핑, 무의미 텍스트	T1195.001
MET-005	Decoy Functionality	MEDIUM	위장 기능으로 악성 행위 은폐	T1195.001
MET-006	Metadata Typosquatting	HIGH	유명 패키지와 편집거리 <= 2	T1195.001

5.8 DOW - Downloader Pattern (다단계 다운로더, 2개)

코드	이름	심각도	설명	MITRE TTP
DOW-001	Single-file Downloader-Exec	HIGH	fetch -> exec/eval 직결	T1059, T1105
DOW-002	Write-then-Exec Downloader	HIGH	fetch -> write 파일 -> 실행	T1105, T1059

6. 6 공격 시퀀스 패턴 (Sequence Pattern)

[src/pkgsentinel/stages/sequence_patterns.py](#). 단일 호출이 아닌 호출 순서를 본다.

코드	이름	심각도	슬롯 (min~max 반복)	MITRE TTP
SP-001	Credential exfiltration	HIGH	INFO_READ(1~5) -> ENCODING(0~3) -> DATA_TX(1~2)	T1552.001, T1041, T1048.003
SP-002	Download-and-execute	HIGH	DATA_TX(1~2) -> EXECUTION(1~2)	T1105, T1059
SP-003	Encoded payload execution	HIGH	ENCODING(1~3) -> EXECUTION(1~2)	T1027, T1059, T1140
SP-004	System reconnaissance + exfil	MEDIUM	INFO_READ(2~10) -> DATA_TX(1~2)	T1082, T1057, T1041
SP-005	Info-driven execution	MEDIUM	INFO_READ(1~3) -> EXECUTION(1~2)	T1059, T1106
SP-006	Full kill-chain	HIGH	INFO_READ -> ENCODING -> EXECUTION -> DATA_TX	T1059, T1041, T1027

탐욕(greedy) 매칭: 각 슬롯을 가능한 한 max 만큼 채운 뒤 다음 슬롯 검사. 슬롯 사이에 다른 차원 호출이 끼면 매칭

실패. 한 파일에서 같은 패턴은 1회만 보고.

7. 21단계 파이프라인

`src/pkgssentinel/pipeline.py` 의 `run_pipeline()`. 입력 1건당 다음 21 stage 가 순차/조건부 실행.

#	Stage ID	모듈	하는 일
0	preflight_llm_check	pipeline.py	LLM API 키/모델 가용성 점검
1	stage_0_registry	stages/stage0_registry.py	PyPI/npm API 메타데이터 조회 (등록 여부, 등록일, 버전 수, repo URL)
2	stage_0a_threat_filter	stages/stage0_threat_filter.py	OSSF malicious-packages DB 와 이름 매칭 (이미 알려진 악성?)
3	stage_0b_attack_history	stages/stage0b_attack_history.py	동명/유사명 과거 공격 기록 조회
4	stage_0c_scorecard	stages/stage_scorecard.py	OpenSSF Scorecard (오픈소스 보안 점수) 조회
5	stage_0d_slsa	stages/stage_slsa.py	SLSA(공급망 빌드 출처 표준) provenance 검증
6	stage_0e_cache_lookup	db/analysis_cache.py	30일 이내 동일 패키지-버전 분석 결과 캐시 조회
7	stage_1b_full_source	stages/stage1b_full_source.py	tar/zip 메모리 스트리밍으로 소스 트리 추출 (설치 안 함)
8	stage_1c_aislopsq	stages/stage_agentic.py	AILOPSQ Manifest (에이전트 패키지 분류 표준) 검사
9	stage_2_behavior_sequence	stages/stage2_behavior.py	AST 기반 4 차원 API 호출 시퀀스 추출
10	stage_2b_string_analysis	stages/string_analysis.py	문자열 엔트로피, 의심 URL, 호모글리프 검사
11	stage_3b_version_diff	stages/stage3b_full_diff.py	이전 버전과 AST 단위 차이 분석 (사후 변조 탐지)
12	stage_4_ttp_matching	stages/stage4_ttp_match.py	sentence-transformers 임베딩 + MITRE ATT&CK 431 techniques 코사인 유사도
13	stage_4b_anomaly_detection	knowledge/anomaly_baseline.py	popular 패키지 baseline 대비 행위 이상치
14	stage_4c_indicator_matcher	stages/indicator_matcher.py	49 악성 지표 매칭
15	stage_4d_taint_slicing	stages/taint_slicer.py	테인트(오염) 분석 - source(환경변수 등) -> sink(네트워크 등) 데이터 흐름 추적
16	stage_4e_sequence_mining	stages/sequence_patterns.py	6 시퀀스 패턴 매칭
17	stage_5_llm_review	stages/stage5_multi_agent.py	Claude 3-에이전트(semantic/diff/dependency) 합의 LLM 검토
18	stage_6_dependencies	stages/stage_dependency.py	직간접 의존성 그래프 재귀 분석 (옵션)
19	stage_7_binary	stages/stage_binary.py	변들 바이너리(.so/.dll/.exe) 메타데이터 분석
20	stage_8_sandbox	stages/stage_sandbox.py	Docker strace 동적 분석 (옵션, SUSPICIOUS만)
21	verdict_decision	verdict_rules.py	Evidence 집계 -> 5단계 판정

7.1 필수 vs 옵션

- 필수 (실패 시 **ERROR**): stage_2_behavior_sequence, stage_4_ttp_matching, stage_5_llm_review
- 조건부: 0c/0d/0e 캐시 히트 시 일부 스킵, 6/7/8은 `--deps` `--sandbox` 플래그
- 차단형(게이트): 1번 stage_0_registry 미등록 -> 즉시 CANNOT_ANALYZE

8. AISLOPSQ Manifest (LLM 에이전트 패키지 분류 표준)

`src/pkgssentinel/agentic/`. pkgssentinel이 발표할 자체 표준.

8.1 동기

LangChain/LlamaIndex/AutoGen 같은 LLM 에이전트 라이브러리는 정상적으로 `exec`, `subprocess`, `network` 를 사용해야 한다. 기존 49 지표로 보면 모두 MALICIOUS로 잘못 잡힐 위험. AISLOPSQ는 이런 패키지를 **agentic-by-design** 으로 따로 표시.

8.2 핵심 모듈

파일	역할
agentic/manifest.py	manifest YAML 스펙 (capability/intent/limits 선언)
agentic/classifier.py	패키지가 agentic 인지 자동 분류
agentic/capability_detector.py	exec/network/file/llm-call 능력 자동 탐지
agentic/rule_of_two.py	"Meta Agents Rule of Two" 정책 - 3개 이상 능력 동시 보유 시 위험
agentic/rules.py	agentic-only 위험 룰 (Prompt Injection, Tool Hijacking)
agentic/signals.py	agentic 신호 감지

8.3 새로운 위험 분류

- **Prompt Injection**: 외부 입력이 LLM 프롬프트를 조작
- **Tool Hijacking**: 에이전트의 도구 호출을 가로채 의도와 다른 동작 유도
- **Capability Overlap**: 한 에이전트가 동시에 너무 많은 능력(파일-네트워크-실행) 보유

8.4 verdict

AISLOPSQ 매칭 + 모든 R1-R4 룰 통과 시 verdict = **AGENTIC** (MALICIOUS 아님). 사용자가 명시적으로 opt-in 해야 설치 권장.

8.5 위험 모델 한계 - 보안 통제가 아닌 *투명성 표준*

발표 시 가장 날카로운 질문이 나올 지점. 외부 코드리뷰에서 지적된 근본 한계를 정직하게 답해야 한다.

- manifest 는 **패키지 작성자가 직접** 작성한다. 슬롭스쿼팅/악성 패키지에서는 작성자 = 공격자.
- 따라서 declared/detected diff 는 *누락에 의한 거짓*(under-declaration)만 잡는다. 공격자가 모든 capability 를 선언하면 이 검사는 무력화됨.
- 결론: **manifest 는 투명성 도구이지 단독 보안 통제가 아니다**. 적응적 공격자에 대한 실질 방어는 하부 behavioral 룰(R1-R4) + 비-agentic 경로의 49 지표에서 나온다.
- 진짜 무결성은 서명된 manifest(Sigstore) + 외부 attestation 이 필요하며 v0.1 범위 밖.

8.6 자기선언 면책은 코드 검증이 있을 때만 (M-A)

자기선언 필드가 *심각도만 낮추는* 방향으로 악용되던 문제를 차단했다.

- `session_isolation = true` 선언만으로는 Lethal Trifecta 위반을 완화하지 못한다. 컨텍스트 리셋 시그니처(`verify_session_isolation`)가 코드에 있을 때만 인정.
- `design_patterns.applied`(dual-llm / plan-then-execute / action-selector 등) 선언도 구현 시그니처(`verify_design_patterns`)로 뒷받침될 때만 R1 룰 면책.
- 미검증 선언은 **DECLARED-BUT-UNVERIFIED** 로 기록만 되고 면책 효과 0 -> TOML 한 줄로 HIGH_RISK 를 깎을 수 없음.

8.7 Rule of Two satisfies 비교 (M-B)

`rule_of_two.satisfies` 가 파싱만 되고 판정에 안 쓰이던 죽은 코드였음. `R3_rule_of_two_consistency` 로 구현:

- detected capability 의 ABC 매핑이 declared satisfies 를 초과(과소 선언) -> SUSPICIOUS.
- satisfies = [A, B, C] 전부 선언(스펙 §4 금지) -> HIGH_RISK.

8.8 스키마 검증 + manifest-부재 처리 (Q-5/Q-6)

- **Q-5**: declared capability 는 canonical 15종 어휘로 검증. 오타/쓰레기 문자열은 `declared_set` 에서 제외하고 `unknown_capabilities` 로 보고(영원한 불일치 방지).
- **Q-6**: manifest 부재(현실 패키지 ~100%) 시 dangerous capability 사용을 *즉시 MALICIOUS* 로 단축하던 FP 폭탄 제거. manifest 가 있을 때만(과소 선언) MALICIOUS, 부재 시엔 behavioral 룰이 판정.

8.9 학술 근거 (`docs/aislopsq/papers/`)

- 01: Chhabra survey - LLM 에이전트 공격면 서베이
- 02: Beurer-Kellner - Agentic 보안 설계 패턴
- 03: ToolHijacker
- 04: "Attacker Moves Second" - 에이전트 적대적 분석
- 05: "Meta Agents Rule of Two" - 능력 결합 정책

9. 6 출력 sink (외부 보안 도구 어댑터)

`src/pkgsentinel/realtime/sinks/` + `src/pkgsentinel/stages/stage_vex.py`.

verdict >= SUSPICIOUS 일 때만 발사.

9.1 CycloneDX VEX v1.5 (`stage\_vex.py`)

- **VEX** = *Vulnerability Exploitability eXchange* (취약점 노출가능성 교환 포맷)
- **CycloneDX** = OWASP 가 만든 SBOM(소프트웨어 부품표) 포맷
- 소비자: Dependency-Track, Anchore, Snyk
- 용도: 기업 SBOM 도구가 어떤 취약점이 실제로 영향 있는지 자동 판단

9.2 STIX 2.1 (`realtime/sinks/stix\_sink.py`)

- **STIX** = *Structured Threat Information eXpression* (구조화 위협 정보 표현, OASIS 표준)
- 위협을 객체 그래프(Indicator/Malware/Campaign/Relationship)로 표현
- 소비자: Splunk, Elastic Security, Sentinel, OpenCTI, MISP

9.3 Falco rules (`realtime/sinks/falco\_policy.py`)

- **Falco** = CNCF(클라우드네이티브재단) 런타임 보안 도구 - syscall 감시 + YAML 룰 매칭
- 소비자: 쿠버네티스/도커 운영자
- 정적 분석에서 발견한 행위 패턴을 *런타임* 차단 룰로 변환

9.4 TAXII 2.1 (`realtime/sinks/taxii\_sink.py`)

- **TAXII** = *Trusted Automated eXchange of Indicator Information* (지표 자동 교환 프로토콜, OASIS 표준)
- STIX 객체를 HTTPS REST로 *주고받는* 전송 프로토콜
- 인증: Basic auth 또는 Bearer (JWT) - OpenCTI/MISP 호환
- 환경변수: `AISLOP_TAXII_URL`, `AISLOP_TAXII_USER/PASS`, `AISLOP_TAXII_BEARER`

9.5 SafeDep pmg policy (`realtime/sinks/pmg\_policy.py`)

- **SafeDep** = 의존성 게이팅 도구, `vet` CLI 제공
- **pmg** = *policy management groups* (정책 그룹)
- 소비자: 개발자 CI 파이프라인 - `vet scan` 이 정책에 걸리면 빌드 실패
- 용도: CI/CD 단계에서 *예방-차단*

9.6 Webhook (`realtime/sinks/webhook\_sink.py`)

- 임의 URL로 HMAC(메시지 인증 코드, 키 기반 서명) 서명된 JSON POST
- 소비자: Slack, Discord, Teams, PagerDuty, n8n, 사내 시스템
- HMAC 서명 헤더: `X-AISLOPSQ-Signature: sha256=... + X-AISLOPSQ-Timestamp`

10. 사후 대응 (Post-Compromise Detection)

분석을 통과한 패키지가 실제로 악성으로 드러나는 케이스 대응.

10.1 버전 위변조 감지 (`stages/stage3\_version\_diff.py`, `stage3b\_full\_diff.py`)

- 신버전 등록 시 이전 버전 N-1, N-3, N-5와 AST 단위 diff
- *추가된 코드*에 대해서만 indicator/sequence 재매칭
- 정상 패키지가 어느 시점부터 악성으로 바뀌는 케이스를 잡음 (event-stream, ua-parser-js 사건)

10.2 SLSA Provenance 검증 (`stages/stage\_slsa.py`)

- **SLSA** = *Supply-chain Levels for Software Artifacts* (공급망 빌드 수준 표준)
- in-toto attestation(빌드 출처 증명) 검증 - 누가 빌드했나, 어떤 커밋에서, 어떤 빌더로
- 이전 버전 SLSA L3 -> 신버전 L0(provenance 없음) = 계정 탈취 의심 신호

10.3 런타임 알람 피드백 (`api/runtime\_alert.py`, `db/runtime\_intel.py`, `intel/extractor.py`)

```
Falco/Tetragon/Wazuh —[HMAC webhook]-> pkgsentinel /api/v1/runtime-alert
|
├─ HMAC + replay 검증 (auth.py)
├─ source별 event 파싱
├─ RuntimeIntelStore 적재
├─ IOC 추출 + auto_promote
└─ 패턴 추출 - novel 이면 룰 draft 생성
```


↳ 패키지 재평가 trigger (옵션)

- 운영 호스트에서 발견된 IOC(*Indicator of Compromise* - 침해 지표) 가 자동 학습됨
- 새 IOC 임계치 도달 시 *학습된 IOC*로 승격, 다른 패키지 분석 시 참조
- 이전 CLEAN 판정 패키지의 verdict 가 retroactively 갱신될 수 있음

10.4 의존성 트리 역추적 (`stages/stage\_dependency.py`)

- 한 패키지가 MALICIOUS로 뒤집히면, 그것에 의존하는 다른 패키지 verdict 도 재계산
- transitive(간접) 의존성까지 추적

10.5 DB 무결성 (`db/integrity.py`, `db/master\_key.py`)

- SQLCipher(SQLite + AES-256 페이지 암호화)
- 행 단위 HMAC 체인 - 한 줄만 바뀌도 검증 실패
- 마스터 키: `/etc/pkgsentinel/env` (mode 0640) 또는 외부 KMS

11. 운영 토폴로지 (Operational Topology)

`deploy/systemd/` 5 service + 3 timer.

```
refresh-feeds.timer (daily 03:30 UTC)
↳ refresh-feeds.service    OSV/GHSA/MITRE 캐시 갱신

watch-pypi.timer (5min)
↳ watch-pypi.service       PyPI XMLRPC -> priority queue 적재

watch-npm.timer (5min)
↳ watch-npm.service        npm changes feed -> priority queue 적재

pkgsentinel-worker.service    queue 상시 consume -> 분석 -> sink 발송
                               (Type=simple, --loop, --llm-model haiku, MemoryMax=4G)

pkgsentinel-server.service    HTTP API - /analyze, /runtime-alert, /iocs-export
                               (gunicorn -w 4 --threads 2, port 8787)
```

11.1 24시간 운영 비용 (Haiku 모드)

- watch-pypi/npm: XMLRPC/changes feed = \$0
- worker: Claude Haiku × 3-agent × ~1000 pkg/day ~ **\$24-45/day**
- refresh-feeds: 네트워크만 = \$0
- Sonnet 모드 시 약 5배 -> \$120-225/day

11.2 환경변수 (`/etc/pkgsentinel/env`)

키	용도	필수
AISLOP_DB_KEY	SQLCipher 마스터 패스프레이즈	필수
ANTHROPIC_API_KEY	Stage 5 Claude API	필수
PKGSENTINEL_HMAC_SECRET	HTTP API 클라이언트 인증	권장
AISLOP_STIX_OUT_DIR	STIX 출력 디렉토리	옵션
AISLOP_FALCO_OUT_DIR	Falco 룰 출력 디렉토리	옵션
AISLOP_WEBHOOK_URL / _SECRET	Webhook 대상 + HMAC 비밀	옵션
AISLOP_PMG_OUT_DIR	SafeDep pmg 정책 출력	옵션
AISLOP_TAXII_URL / _USER / _PASS / _BEARER	TAXII 서버	옵션

11.3 보안 격리

- `User=pkgsentinel` 비특권 service account
- `NoNewPrivileges=true`, `ProtectSystem=strict`, `ProtectHome=true`, `PrivateTmp=true`
- 악성 패키지 소스 코드 *자체는 실행 안 함* - 정적 분석만. 동적 분석은 별도 Docker 컨테이너.

12. 데이터베이스 구조

12.1 위협 DB (`threat\_db.sqlcipher`)

`src/pkgsentinel/db/threat_db.py`. SQLCipher 단일 파일.

테이블	용도
analyses	패키지-버전별 verdict, evidence, 분석 시각
known_malicious	OSSF malicious-packages + 학습된 악성
known_popular	인기 패키지 baseline (FP 억제용)
network_blocklist	의심 도메인/IP
feed_meta	피드 갱신 메타데이터 (마지막 갱신 시각 등)
stage_cache	stage별 부분 캐시 (db/stage_cache.py)
cache_invalidation_log	캐시 무효화 이력

12.2 분석 캐시 (`db/analysis\_cache.py`)

같은 패키지-버전을 반복 분석하지 않음. TTL 30일.

12.3 런타임 인텔 (`db/runtime\_intel.py`)

Falco/Tetragon/Wazuh 로부터 받은 관측 + 학습된 IOC.

테이블	용도
runtime_observations	원본 이벤트 적재
learned_iocs	auto_promote 된 IOC
pattern_drafts	새 패턴 룰 초안

12.4 우선순위 큐 (`monitor/priority\_queue.py`)

`scan_queue.sqlcipher`. watcher 가 적재, worker 가 pop.

우선순위 기준:

- 등록 30일 이내
- popular 패키지와 편집거리 ≤ 2
- 버전 1개뿐
- repo URL 부재

12.5 지식 캐시 (`knowledge/cache/`)

파일	출처	크기
osv_pypi.json	OSV.dev PyPI advisory	~150MB
osv_npm.json	OSV.dev npm advisory	~100MB
mitre_attack.json	MITRE ATT&CK Enterprise	~10MB
mitre_attack_embedded.json	위 + sentence-transformer 임베딩	~30MB

자동 갱신: `pkgsentinel-refresh-feeds.timer` daily 03:30 UTC.

13. 지식 베이스 (Knowledge Base)

`src/pkgsentinel/knowledge/`.

모듈	데이터	용도
attack_index.py	MITRE ATT&CK 431 techniques + sub-techniques	코사인 유사도 인덱스
embedder.py	sentence-transformers all-MiniLM-L6-v2	코드/시퀀스 임베딩
mitre_attack.py	MITRE ATT&CK Enterprise Matrix	TTP 데이터
mitre_atlas.py	MITRE ATLAS - AI/LLM 공격 TTP	에이전트 위협 매핑
owasp_llm.py	OWASP Top 10 for LLM (LLM 특화 취약점)	LLM 위협 매핑
osv.py	OSV.dev (Open Source Vulnerabilities) PyPI+npm 22만+ advisory	알려진 취약점
ossf_malicious.py	OSSF malicious-packages DB	알려진 악성
ossf_package_analysis.py	OSSF 패키지 분석 결과	추가 신호
anomaly_baseline.py	인기 패키지 행위 baseline	FP 억제
package_baseline.py	패키지별 baseline 캐시	FP 억제
malicious_indicators.py	49 지표 카탈로그 (섹션 5)	매칭 룰

13.1 외부 데이터 소스 요약

소스	종류	갱신
MITRE ATT&CK	사이버 공격 기법 분류 표준	github.com/mitre/cti pull
MITRE ATLAS	AI/ML 시스템 공격 분류	ATLAS 공식 JSON
OWASP LLM Top 10	LLM 보안 위협 카테고리	공식 자료 파싱
OWASP Top 10 (Web)	일반 웹 취약점	참고용
NIST SSDF v1.1	보안 소프트웨어 개발 프레임워크	stages/stage_ssdf.py

소스	종류	갱신
SLSA v1.0	공급망 빌드 수준 표준	stages/stage_slsa.py
OSV.dev	오픈소스 취약점 DB (구글)	API
GitHub Advisory (GHSA)	공급망 공격 공개 사례	GHSA 피드
OpenSSF Scorecard	오픈소스 보안 점수	API
OSSF malicious-packages	알려진 악성 패키지 DB	git pull

14. 입출력 인터페이스

14.1 CLI (`src/pkgsentinel/cli.py`)

pkgsentinel <package> [옵션]

옵션:

```
--ecosystem, -e {PyPI, npm}   레지스트리 선택 (기본 PyPI)
--version, -v VERSION         특정 버전 (기본 latest)
--llm {stub, claude}          Stage 5 모드 (기본 claude)
                                stub = 정적 분석만, FP 올 매우 높음
                                claude = ANTHROPIC_API_KEY 필요
--deps                        의존성 재귀 분석
--sandbox                     Docker strace 동적 분석
--json                        JSON 출력 (기본은 사람 읽기 형식)
```

콘솔 스크립트 4개 (pyproject.toml):

- **pkgsentinel** - 단일 패키지 분석
- **pkgsentinel-feeds** - 위협 피드 일괄 갱신
- **pkgsentinel-worker** - 큐 상시 consume 워커
- **pkgsentinel-cron** - watch-pypi/watch-npm/refresh-feeds 트리거

14.2 HTTP API (`src/pkgsentinel/server/app.py`)

FastAPI/Flask 호환. gunicorn -w 4 --threads 2 포트 8787.

엔드포인트	메소드	용도
/api/v1/analyze	POST	패키지 분석 요청
/api/v1/runtime-alert	POST	Falco/Tetragon/Wazuh 알람 수신
/api/v1/iocs/export	GET/POST	학습된 IOC 내보내기
/healthz	GET	liveness probe
/readyz	GET	readiness probe
/metrics	GET	Prometheus 메트릭

인증: HMAC-SHA256 (auth.py), 헤더 X-AISLOPSQ-Signature, X-AISLOPSQ-Timestamp. Replay 방지(5분 윈도우 + nonce LRU).

보안 강화 (외부 리뷰 대응 C-1/C-2):

- **fail-closed:** PKGSENTINEL_HMAC_SECRET 미설정 + dev opt-in(PKGSENTINEL_DEV_NO_AUTH=1) 없으면 보호 endpoint 가 503 반환(구버전은 무인증 통과). 기본 bind 127.0.0.1(loopback).
- **iocs/export GET 도 인증:** 구버전은 GET 시 HMAC 을 전면 우회해 학습된 위협 인텔이 무인증 유출됐음. 이제 GET 은 원본 query string 에 서명 검증.

14.3 Chrome Extension (MV3)

- claude.ai / chatgpt.com / gemini.google.com 응답을 MutationObserver(DOM 변화 감지 API)로 실시간 감시
- 코드블록 추출 -> entrypoint/import_parser.py 로직으로 패키지명 파싱
- 로컬 HTTP API 호출 -> 결과를 인라인 배지/경고 패널로 표시

14.4 VSCode Extension

- 파일 저장 시 import 문 파싱
- 로컬 HTTP API 호출
- 의심 import 위에 진단(diagnostic) 표시

15. 디렉토리 전체 구조

pkgsentinel/

```

└─ pyproject.toml          패키지 메타데이터, 4 console scripts
└─ README.md
└─ CONTRIBUTING.md
└─ SECURITY.md
└─ LICENSE                Apache-2.0
|
└─ src/pkgsentinel/
|   └─ __init__.py
|   └─ __main__.py        python -m pkgsentinel 진입점
|   └─ cli.py             CLI 진입점
|   └─ pipeline.py        21 stage 메인 파이프라인
|   └─ schema.py          Verdict/Severity/Evidence 등 데이터 모델
|   └─ verdict_rules.py   5단계 판정 규칙
|   └─ _dotenv.py         .env 파일 로더
|   └─ _pipeline_state.py stage 간 컨텍스트 전달
|   |
|   └─ stages/            21 stage 구현
|       └─ stage0_registry.py
|       └─ stage0_threat_filter.py
|       └─ stage0b_attack_history.py
|       └─ stage1_entry_point.py
|       └─ stage1b_full_source.py
|       └─ stage2_behavior.py
|       └─ stage3_version_diff.py
|       └─ stage3b_full_diff.py
|       └─ stage4_rules.py
|       └─ stage4_ttp_match.py
|       └─ stage5_llm_review.py    deprecated, multi_agent 가 대체
|       └─ stage5_multi_agent.py   Claude 3-에이전트 합의
|       └─ stage_agentic.py        AISLOPSQ Manifest 검사
|       └─ stage_binary.py         번들 .so/.dll/.exe 메타데이터
|       └─ stage_dependency.py     의존성 그래프 재귀
|       └─ stage_sandbox.py        Docker strace 동적 분석
|       └─ stage_scorecard.py      OpenSSF Scorecard
|       └─ stage_slsa.py           SLSA provenance
|       └─ stage_ssdf.py           NIST SSDF 매핑
|       └─ stage_vex.py            CycloneDX VEX 출력
|       └─ api_catalog.py          4 차원별 API 매핑 카탈로그
|       └─ deobfuscator.py         base64/hex/문자열 난독화 해제
|       └─ indicator_matcher.py    49 지표 매칭 로직
|       └─ js_ast_parser.py        tree-sitter JavaScript 파서
|       └─ sequence_patterns.py    6 시퀀스 패턴 + 탐욕 매칭
|       └─ string_analysis.py      엔트로피/URL/호모글리프
|       └─ taint_slicer.py         테인트 분석 (source -> sink)
|   |
|   └─ knowledge/           지식 베이스
|       └─ attack_index.py        431 TTP 코사인 유사도 인덱스
|       └─ embedder.py            sentence-transformers
|       └─ mitre_attack.py
|       └─ mitre_atlas.py
|       └─ owasp_llm.py
|       └─ osv.py
|       └─ ossf_malicious.py
|       └─ ossf_package_analysis.py
|       └─ anomaly_baseline.py
|       └─ package_baseline.py
|       └─ malicious_indicators.py 49 지표 카탈로그
|       └─ cache/              자동 갱신되는 JSON 캐시
|   |
|   └─ agentic/             AISLOPSQ Manifest 구현
|       └─ manifest.py

```

			└ classifier.py	
			└ capability_detector.py	
			└ rule_of_two.py	
			└ rules.py	
			└ signals.py	
			└ db/	데이터베이스 계층
			└ threat_db.py	메인 SQLCipher DB
			└ analysis_cache.py	분석 결과 캐시 (TTL 30일)
			└ stage_cache.py	stage별 부분 캐시
			└ runtime_intel.py	런타임 관측/IOC
			└ master_key.py	AES-256 마스터 키 관리 (env/KMS)
			└ integrity.py	행 단위 HMAC 체인
			└ monitor/	실시간 감시
			└ cron_main.py	cron endpoint
			└ pypi_watcher.py	PyPI XMLRPC changelog
			└ npm_watcher.py	npm CouchDB changes feed
			└ priority_queue.py	우선순위 큐
			└ release_event.py	release 이벤트 데이터 모델
			└ worker.py	큐 consumer
			└ feeds/	외부 피드 수집
			└ osv.py	OSV.dev advisory pull
			└ popular.py	인기 패키지 목록
			└ network_ioc.py	네트워크 IOC 피드
			└ refresh.py	통합 갱신
			└ intel/	IOC 인텔리전스
			└ extractor.py	이벤트에서 IOC 추출
			└ rule_generator.py	novel 패턴 -> 룰 draft
			└ osv_export.py	학습 IOC를 OSV 형식으로 export
			└ cli.py	
			└ realtime/	실시간 sink
			└ sinks/	
			└ stix_sink.py	STIX 2.1
			└ taxii_sink.py	TAXII 2.1 push
			└ falco_policy.py	Falco YAML
			└ pmg_policy.py	SafeDep pmg
			└ webhook_sink.py	HMAC webhook
			└ evidence/	Evidence 변환
			└ converters.py	indicator/sequence -> Evidence
			└ snippets.py	코드 스니펫 추출
			└ reporting/	리포트 생성
			└ formats.py	사람 읽기/JSON 포맷
			└ serialize.py	
			└ server/	HTTP API 서버
			└ app.py	Flask/FastAPI app
			└ __main__.py	gunicorn endpoint
			└ api/	API 핸들러
			└ analyze.py	/api/v1/analyze
			└ runtime_alert.py	/api/v1/runtime-alert
			└ iocs_export.py	/api/v1/iocs/export
			└ auth.py	HMAC 검증
			└ endpoint/	클라이언트용 헬퍼

```

|   |   └ import_parser.py      AST + Regex 하이브리드 import 파서
|   |
|   └ benchmarks/              내부 벤치마크 하네스
|       └ harness.py
|
└ tests/                        40+ 단위/통합 테스트
└ scripts/                      평가/도구 스크립트
    └ eval_synthetic.py        합성 악성 패턴 정확도 평가
    └ eval_popular9.py         인기 패키지 FP 측정
    └ eval_real.py             실제 패키지 평가
    └ eval_real_fetch.py / _analyze.py / _compare.py / _resynth.py
    └ eval_deps_recursion.py
    └ eval_deps_full_recursion.py
    └ eval_llm_consistency.py   LLM 일관성 측정
    └ audit_gate.py
    └ demo_capstone_dry_run.py
    └ monitor_smoke.py / monitor_extended_smoke.py
    └ npm_watcher_smoke.py
    └ sink_e2e_demo.py
    └ gen_system_diagram.py     시스템 구조도 PNG 생성
    └ gen_week11_pdf.py        주차 보고서 PDF
|
└ docs/
    └ architecture.md          설계 문서
    └ cost_model.md            LLM 비용 모델
    └ 2026-04-29-*.md          초기 설계/평가
    └ 2026-05-06-project-review.md  중간 감사
    └ 2026-05-13-academic-impact-map.md  학술 근거 매핑
    └ 2026-05-18-week11-progress.md  11주차 보고서
    └ 2026-05-19-booklet.md      발표 책자
    └ aislopsq/                 AISLOPSQ 스펙 + 학술 근거
        └ spec/AISLOPSQ-MANIFEST-SPEC.md
        └ detection/AGENTIC-SIGNALS.md
        └ detection/CAPABILITY-DETECTION.md
        └ papers/01~05
    └ references/               참고문헌 (54건)
    └ diagrams/
|
└ deploy/
    └ systemd/                 5 service + 3 timer + README
    └ docker/                  docker-compose.yml + README
    └ fim/                     파일 무결성 모니터 자산

```

16. 테스트 커버리지

tests/ 40+ 파일.

영역	테스트 파일
Schema 일관성	test_schema_compliance.py
Verdict 규칙	test_verdict_rules.py
지표 매칭	test_indicator_evidence.py (지표 데모는 examples/indicator_47_demo.py)
시퀀스 매칭	test_sequence_patterns.py
테인트 분석	test_taint_slicer.py
AISLOPSQ	test_agentic.py
멀티 에이전트	test_multi_agent.py
의존성	test_dependency_parser.py, test_deps_transitive.py, test_dep_range_resolution.py, test_max_satisfying.py
출력 sink	test_vex.py, test_pmg_sink.py, test_taxii_sink.py, test_webhook_sink_post.py
표준 매핑	test_atlas_owasp.py, test_slsa.py, test_ssdf.py, test_scorecard.py

영역	테스트 파일
외부 데이터	test_ossf_malicious.py, test_ossf_package_analysis.py
위협 DB	test_threat_db_integration.py, test_stage_cache.py, test_master_key_kms.py
런타임 인텔	test_runtime_alert_api.py, test_runtime_intel_store.py, test_realtime_pipeline.py
HTTP API	test_api_analyze_and_iocs.py, test_api_auth.py, test_server_flask.py
워커	test_worker_llm_model.py
평가 하네스	test_benchmark_harness.py
합성 악성	scripts/eval_synthetic.py + examples/synthetic_malicious_demo.py (pytest 파일 아님, 수동 실행)
동적 분석	test_sandbox_strace.py
난독화	test_z1_deobfuscator.py
다운로더 패턴	test_r1_strict_mode_and_z2_downloader.py
변종 진단	test_z3_baseline_and_z6_dep_diff.py
FIM 자산	test_r5_fim_assets.py
stage3b 캐시	test_stage3b_cache_load.py
binary	test_stage_binary.py
TTP 인덱스	test_attack_index_version.py
Intel	test_intel_osv_export_and_cli.py, test_intel_rule_generator.py

현재 전체 **398개 통과** (외부 코드리뷰 대응으로 회귀 테스트 13개 추가 - 서버 fail-closed/GET 인증 5, manifest 면책-satisfies 5, MALICIOUS max-gate 1, taint with-as 1, Q-롤 1).

CI 구성 ([.github/workflows/ci.yml](#)):

- Ubuntu/Windows × Python 3.11/3.12 4매트릭스
- ruff blocking lint
- pip-audit high/critical gating
- CodeQL static analysis

17. 외부 표준 매핑

표준	종류	pkgsentinel 위치
MITRE ATT&CK	사이버 공격 기법 분류 (Enterprise Matrix)	knowledge/mitre_attack.py, 431 techniques
MITRE ATLAS	AI/ML 시스템 공격 기법 분류	knowledge/mitre_atlas.py
OWASP LLM Top 10	LLM 보안 위협 카테고리	knowledge/owasp_llm.py
NIST SSDF v1.1	Secure Software Development Framework	stages/stage_ssdf.py
SLSA v1.0	Supply-chain Levels for Software Artifacts	stages/stage_slsa.py
OSV.dev	Open Source Vulnerabilities DB (Google)	knowledge/osv.py, feeds/osv.py
GitHub Advisory (GHSA)	공급망 공격 공개 사례	OSV 경유
OpenSSF Scorecard	오픈소스 보안 점수	stages/stage_scorecard.py
OSSF malicious-packages	알려진 악성 패키지 DB	knowledge/ossf_malicious.py
CycloneDX VEX v1.5	SBOM 취약점 어노테이션	stages/stage_vex.py
STIX 2.1	위협 정보 구조화 표현	realtime/sinks/stix_sink.py
TAXII 2.1	위협 정보 전송 프로토콜	realtime/sinks/taxii_sink.py
Falco	CNCF 런타임 보안 룰	realtime/sinks/falco_policy.py
SafeDep pmg	의존성 게이팅 정책	realtime/sinks/pmg_policy.py

18. 평가 지표 (scripts/eval_*.py)

18.1 측정 방법

- **합성 악성** (eval_synthetic.py): 49 지표 + 6 시퀀스 패턴마다 양성 샘플 자동 생성 -> 탐지율 측정
- **인기 패키지 FP** (eval_popular9.py): top-N 인기 패키지를 분석 -> SUSPICIOUS 이상이 나오면 FP
- **실제 패키지** (eval_real.py): OSSF malicious-packages + 일반 PyPI/npm 표본 비교
- **LLM 일관성** (eval_llm_consistency.py): 같은 입력에 대한 multi-agent 합의 일관성

18.2 주요 측정 결과 (11주차 보고서 기준)

지표	값
합성 악성 Recall (재현율)	0.98 -> 0.87 회귀 발견 -> 0.98 복구 (commit dc4df27)
popular 9 패키지 FP	category-aware STANDALONE_WEAK 게이트로 16% -> 측정값 관리

지표	값
LLM 합의 일관성	3-agent quorum 2/3 매칭률 측정
21-stage 평균 실행 시간	Haiku ~8s/pkg, Sonnet ~25s/pkg

19. 한계 (정직한 약점)

한계	설명	완화
고도 난독화	marshalling, AES 암호화된 페이로드는 정적 분석으로 보기 어려움	Stage 8 sandbox 옵션 (Docker strace)
새로운 공격 기법	49 지표-6 시퀀스 밖의 novel attack	Stage 5 LLM이 semantic 으로 잡음 + runtime feedback 으로 학습
LLM 환각	Claude 자체의 false alarm	3-agent 합의 + STANDALONE_WEAK 게이트 + LLM=BENIGN 우선 정책
SLSA provenance 부재	메인테이너가 발급 안 하면 신호 없음	부재 자체를 약한 의심 신호로만 사용
호스트 EDR 없으면 사후 약함	런타임 피드백은 Falco/Tetragon/Wazuh 설치 전제	사전 검사로 99% 차단, EDR은 보조
학생 환경 비용	Sonnet 풀모드 \$120-225/day	Haiku 기본, 우선순위 큐로 worker 처리량 제어
의존성 점진적 변조	한 번에 안 바꾸고 여러 버전에 쪼개 넣음	cumulative diff (N-1 + N-3 + N-5 동시 비교)
테인트 flow-insensitive	함수 스코프/순서 무시. with open() as f 미seed 었음	부분 해결: with-as seed + tainted receiver 메서드 전파(data=f.read()). 본체 flow-sensitive 재작성은 미완
eval 스크립트-LLM 경로 CI 미포함	eval_*-LLM 합의는 네트워크/API 키 필요	CI 는 unit 만; 수동 실행

20. 용어집 (가나다순)

용어	풀이
AISLOPSQ	AI Slopsquatting 약어, pkgssentinel이 제안하는 에이전트 패키지 분류 표준
AST	Abstract Syntax Tree, 추상구문트리. 코드를 노드 트리로 표현한 것
CycloneDX	OWASP 의 SBOM(소프트웨어 부품표) 포맷
EDR	Endpoint Detection and Response. 엔드포인트 보안 도구
Evidence	pkgssentinel의 판정 근거 단위. 점수가 아닌 구조화된 근거
Falco	CNCF의 런타임 보안 도구. syscall + YAML 룰
FIM	File Integrity Monitoring, 파일 무결성 모니터링
FP	False Positive, 거짓 양성. 정상울 악성으로 잘못 판정
FN	False Negative, 거짓 음성. 악성을 정상으로 놓침
GHSA	GitHub Security Advisory
HMAC	Hash-based Message Authentication Code. 키 기반 메시지 인증 코드
IOC	Indicator of Compromise, 침해 지표
KMS	Key Management Service. AWS/GCP의 키 관리 서비스
LLM	Large Language Model, 거대 언어 모델 (Claude/GPT/Gemini 등)
MITRE ATT&CK	사이버 공격 기법 분류 프레임워크 (Enterprise/ICS/Mobile Matrix)
MITRE ATLAS	AI/ML 시스템 공격 기법 분류 프레임워크
MV3	Manifest V3, Chrome Extension 최신 명세
OASIS	STIX/TAXII 표준을 관리하는 표준화 기구
OSSF	Open Source Security Foundation
OSV	Open Source Vulnerabilities. Google이 만든 오픈소스 취약점 DB
OWASP	Open Web Application Security Project. 웹 보안 표준 기구
OWASP LLM Top 10	LLM 특화 보안 위협 10대 카테고리
Provenance	빌드 출처 증명. SLSA 표준이 정의
SBOM	Software Bill of Materials, 소프트웨어 부품표
SCA	Software Composition Analysis, 의존성 분석 도구류
SIEM	Security Information and Event Management. 보안 이벤트 관리 시스템
SLSA	Supply-chain Levels for Software Artifacts
SOAR	Security Orchestration, Automation and Response
SP-001~006	pkgssentinel의 6개 공격 시퀀스 패턴 코드
SQLCipher	SQLite + AES-256 페이지 암호화

용어	풀이
SSDF	Secure Software Development Framework. NIST가 정의
STIX	Structured Threat Information eXpression. 위협 정보 구조화 표준
Taint Analysis	테인트(오염) 분석. source -> sink 데이터 흐름 추적
TAXII	Trusted Automated eXchange of Indicator Information
Tetragon	Cilium의 eBPF 기반 런타임 보안 도구
TIP	Threat Intelligence Platform. 위협 인텔 플랫폼
TTL	Time To Live. 캐시 유효 기간
TTP	Tactics, Techniques, Procedures. 공격자의 전술/기법/절차
VEX	Vulnerability Exploitability eXchange. 취약점 노출가능성 어노테이션
Wazuh	오픈소스 SIEM/XDR 플랫폼
tree-sitter	점진적 파서 라이브러리. 다언어 AST 생성에 사용
sentence-transformers	문장 임베딩 라이브러리. all-MiniLM-L6-v2 모델 사용
콤보스쿼팅	유명 패키지명 + 단어 결합 (예: requests-utils)
타이포스쿼팅	유명 패키지명의 오타 변종 (예: requets)
호모글리프	모양만 닮은 글자 (예: 0/O, l/I/1)
슬롭스쿼팅	LLM 환각으로 만들어진 가상 패키지명을 공격자가 선등록하는 공격
의존성 혼동	사내 패키지명을 퍼블릭 레지스트리에 더 높은 버전으로 올려 잘못 설치 유도

21. 자주 나올 만한 Q&A

Q1. 다른 SCA 도구(Snyk, Dependabot)와 차이? A. 기존 SCA는 advisory DB(OSV/GHSA)에 등록된 *알려진* 취약점만 본다. pkgscintinel은 advisory 없이도 *행위-메타데이터-의미*로 판정 -> zero-day 윈도우(등록 ~ advisory 등재까지 약 1주)를 메움.

Q2. LLM 환각 패키지인지 어떻게 가려내나? A. (1) 레지스트리 메타데이터 - 등록 30일 이내, 버전 1개, repo URL 부재. (2) 패키지명 - 유명 패키지명과 편집거리 <= 2 (MET-006). (3) 소스 행위 - 49 지표 매칭. (4) AI 추천 컨텍스트 (Extension 측에서 같은 패키지명이 여러 AI 응답에 반복 등장하는지). 단일 신호가 아닌 *결합*.

Q3. False Positive 어떻게 줄이나? A. (1) STANDALONE_WEAK 게이트 - Path.home 단독은 SUSPICIOUS 안 올림. (2) BROAD_PURPOSE 카테고리 패키지(설치 도구-변들러 등)는 별도 baseline. (3) LLM=BENIGN 우선 정책. (4) LLM_SUSPICIOUS_QUORUM=2 - LLM 단독으로 SUSPICIOUS 못 만듦. (5) 인기 패키지 anomaly_baseline 차감. (6) manifest 부재 시 dangerous capability 를 즉시 MALICIOUS 로 단축하지 않음(Q-6) - 정직한 LangChain 에이전트가 manifest 없이 subprocess 써도 즉시 악성 판정 안 됨. (7) agentic 룰 FP 교정 - getattr 디스패치/벡터스토어/widget-target 함수명 오분류 제거.

Q4. 왜 설치 안 하고 분석? A. 악성 패키지의 install hook(setup.py cmdclass)이 분석기 자체를 감염시키는 시나리오 차단. tarball/wheel을 메모리에서 직접 파싱 - 코드는 *읽기만*, 실행 0.

Q5. Claude 한 회사에만 의존? A. Stage 5 multi-agent 는 *같은 Claude 모델*을 3개 *페르소나*(semantic/diff/dependency)로 병렬 호출 -> 다수결. 모델 추상화(stage5_multi_agent.py) 는 Anthropic 외에도 확장 가능하지만 현재 기본은 Claude 단일 vendor. 비용-일관성 trade-off.

Q6. 분석 1건당 비용? A. Haiku(저비용 모델) 기준 3-agent x 평균 3000 토큰 = 약 \$0.01/건. Sonnet은 약 \$0.05/건. Worker 일평균 800-1500 패키지 처리 = \$24~45/day (Haiku).

Q7. 처리 속도? A. Haiku 모드 평균 ~8초/패키지 (소스 인출 + 21 stage). Sonnet 모드 ~25초. Stage 5 LLM 호출이 병목. 캐시 히트 시 즉시 반환.

Q8. event-stream 사건 재현 가능? A. Stage 3b version_diff 가 이전 버전 N-1, N-3, N-5 와 AST diff -> 신규 추가된 requests.post + 외부 도메인 상수를 잡음. *인기도 무시 원칙* 으로 popular 패키지도 풀 파이프라인 실행.

Q9. 데이터 프라이버시? A. (1) 패키지 소스만 분석 - 사용자 데이터 미사용. (2) Stage 5 LLM 호출 시 Anthropic 에 코드 스니펫만 전송 (메타데이터 제외). (3) 로컬 SQLCipher DB AES-256. (4) 사내 배포 시 Anthropic 대신 Claude on AWS Bedrock / Azure 로 격리 가능.

Q10. CI/CD 통합 방법? A. (1) pkgscintinel <pkg> CLI 를 build script 에 추가, exit code != 0 시 fail. (2) SafeDep pmg 정책 출력 -> vet scan 으로 자동 게이팅. (3) HTTP API + 사내 PR 자동화로 의존성 변경 시 자동 검사.

Q11. 환각 데이터셋은? A. research/ 모듈이 GPT-4o / Claude / Gemini 에 500+ 코딩 질문을 던져 응답에서 import 문 파싱 -> PyPI/npm 등록 여부 확인 -> 환각률 측정. 별도 프로젝트로 분리 예정.

Q12. 향후 계획? A. (1) AISLOPSQ Manifest 표준화 제안. (2) Chrome Extension 안정화 (claude.ai/chatgpt/gemini 응답 실시간 검사). (3) 의존성 그래프 시각화. (4) 다른 레지스트리 확장 (Maven, RubyGems, crates.io). (5) manifest 무결성을 위한 Sigstore 서명 + 외부 attestation.

Q13. 최근(외부 코드리뷰 대응) 무엇을 고쳤나? A. 외부 코드리뷰를 받아 검증->수정->재대조 사이클로 처리(docs/2026-05-19-review-checklist.md). 핵심:

- **운영 보안:** HTTP API fail-closed 전환(미인증 통과 차단), iocs-export GET HMAC 우회 차단.
- **manifest:** 자기선언(session_isolation/design_patterns) 면책을 코드 검증 시에만 인정, satisfies 일관성 검사 구현, canonical 어휘 스키마 검증, manifest-부재 FP 폭탄 제거.
- **정확성:** MALICIOUS 게이트 평균->max(저신뢰 잡음에 의한 강등 방지), 테인트 with...as seed, OSV zip-bomb 상한, Docker 샌드박스 cap-drop/pids/memory 격리.
- **문서:** 지표 49/36 표기, "세 겹 무결성"->기본 1겹, manifest 를 투명성 표준으로 정직하게 재서술.
- **결과:** 전체 398 테스트 통과(회귀 13개 추가), main 머지 완료.

문서 끝. (최종 갱신: 외부 코드리뷰 대응 머지 반영)